LABSHEET-8 | Cu25250022 | Aryan

Q1. Generic Box Create a generic class Box that stores one value. Create objects for Integer, String and Double and display all three values.

```
class Box<T> {
   T value;

   Box(T value) {
      this.value = value;
   }

   void display() {
      System.out.println("Value: " + value);
   }
}

public class Q1 {
   public static void main(String[] args) {
      Box<Integer> b1 = new Box<>(100);
      Box<String> b2 = new Box<>("Hello Java");
      Box<Double> b3 = new Box<>(25.5);

      b1.display();
      b2.display();
      b3.display();
   }
}
```

OUTPUT:-

Value: 100

Value: Hello Java

Value: 25.5

Q2. Generic Result Create a generic class Result that stores marks. Create Result and Result objects and display the marks.

```
class Result<T> {
  T marks;

  Result(T marks) {
    this.marks = marks;
  }

  void display() {
    System.out.println("Marks: " + marks);
  }
}

public class Q2 {
  public static void main(String[] args) {
    Result<Integer> r1 = new Result<>(85);
    Result<Double> r2 = new Result<>(92.5);

    r1.display();
    r2.display();
  }
}
```

OUTPUT:-

Marks: 85

Marks: 92.5

Q3. Generic Student Create a generic class Student where T represents the student ID. Create a Student object and display the ID

```
class Student<T> {
  T studentId;

  Student(T studentId) {
    this.studentId = studentId;
  }

  void display() {
    System.out.println("Student ID: " + studentId);
  }
}

public class Q3 {
  public static void main(String[] args) {
    Student<Integer> s1 = new Student<>(101);

    s1.display();
  }
}
```

OUTPUT:-

Student ID: 101

Q4. Generic Pair Create Pair to store Employee ID and Employee Name. Use Integer and String as the two types.

```
class Pair<T, U> {
  T employeeId;
```

```
    U employeeName;

    Pair(T employeeId, U employeeName) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
    }

    void display() {
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Employee Name: " + employeeName);
    }
}

public class {
    public static void main(String[] args) {
        Pair<Integer, String> e =
                new Pair<>(201, "Aarav");

        e.display();
    }
}
```

OUTPUT:-

Employee ID: 201

Employee Name: Aarav

Q5. Product Pair Create a generic class with two type parameters to store Product ID and Product Price. Create at least three product objects.

```
class Product<T, U> {
    T productId;
```

```
    U price;

    Product(T productId, U price) {
        this.productId = productId;
        this.price = price;
    }

    void display() {
        System.out.println("Product ID: " + productId
                + ", Price: " + price);
    }
}

public class Q5 {
    public static void main(String[] args) {
        Product<Integer, Double> p1 =
                new Product<>(1, 499.50);

        Product<Integer, Double> p2 =
                new Product<>(2, 799.00);

        Product<Integer, Double> p3 =
                new Product<>(3, 1250.75);

        p1.display();
        p2.display();
        p3.display();
```

```
    }
}
```

OUTPUT:-

```
Product ID: 1, Price: 499.5
Product ID: 2, Price: 799.0
Product ID: 3, Price: 1250.75
```

Q6. Generic Employee Create Employee where T represents Employee ID and U represents Employee Name. Add a display() method.

```
class Employee<T, U> {
    T employeeId;
    U employeeName;

    Employee(T employeeId, U employeeName) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
    }

    void display() {
        System.out.println("ID: " + employeeId);
        System.out.println("Name: " + employeeName);
    }
}

public class Q6 {
    public static void main(String[] args) {
        Employee<Integer, String> e =
                new Employee<>(301, "Riya");
```

```
        e.display();
    }
}
```

OUTPUT:-

ID: 301

Name: Riya

Q7. Generic Method Create a generic method display() that can display Integer, Double, String and Character values.

```
public class Q7 {

    public static <T> void display(T value) {
        System.out.println(value);
    }

    public static void main(String[] args) {
        display(10);
        display(12.5);
        display("Java");
        display('A');
    }
}
```

OUTPUT:-

10

12.5

Java

A

Q8. Generic Method for Three Values Create a generic method that accepts three values of the same type and displays them. Test it using Integers, Strings and Doubles.

```
public class Q8 {

   public static <T> void displayThree(
         T a, T b, T c) {

      System.out.println(a + ", " + b + ", " + c);
   }

   public static void main(String[] args) {

      displayThree(10, 20, 30);

      displayThree("Red", "Green", "Blue");

      displayThree(1.5, 2.5, 3.5);
   }
}
```

OUTPUT:-

10, 20, 30

Red, Green, Blue

1.5, 2.5, 3.5

Q9. Generic ArrayList Create an ArrayList. Add 10 numbers and display them using a loop.

```
import java.util.ArrayList;

public class Q9 {
   public static void main(String[] args) {
```

```
        ArrayList<Integer> numbers = new ArrayList<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);
        numbers.add(50);
        numbers.add(60);
        numbers.add(70);
        numbers.add(80);
        numbers.add(90);
        numbers.add(100);

        for (int number : numbers) {
            System.out.println(number);
        }
    }
}
```

OUTPUT;-

10

20

30

40

50

60

70

80

90

100

Q10. Generic String List Create an ArrayList containing five student names. Perform add(), get(), set(), remove() and size(), and display the results.

```
import java.util.ArrayList;


public class Q10 {
   public static void main(String[] args) {


      ArrayList<String> names = new ArrayList<>();


      names.add("Aarav");
      names.add("Riya");
      names.add("Karan");
      names.add("Meera");
      names.add("Anaya");


      System.out.println("Original list: " + names);


      System.out.println("Element at index 1: "
            + names.get(1));


      names.set(2, "Kabir");


      System.out.println("After set(): " + names);


      names.remove("Meera");
```

```
        System.out.println("After remove(): " + names);

        System.out.println("Size: " + names.size());
    }
}
```

OUTPUT:-

```
Original list: [Aarav, Riya, Karan, Meera, Anaya]
Element at index 1: Riya
After set(): [Aarav, Riya, Kabir, Meera, Anaya]
After remove(): [Aarav, Riya, Kabir, Anaya]
Size: 4
```

Q11. Search in Generic List Create ArrayList containing ten numbers. Ask the user for a number and use contains() to determine whether the number exists.

```
import java.util.ArrayList;
import java.util.Scanner;

public class Q11 {
    public static void main(String[] args) {

        ArrayList<Integer> numbers = new ArrayList<>();

        Scanner sc = new Scanner(System.in);

        for (int i = 1; i <= 10; i++) {
            numbers.add(i * 10);
        }

        System.out.print("Enter a number to search: ");
```

```
        int search = sc.nextInt();

        if (numbers.contains(search)) {
            System.out.println(search + " exists in the list.");
        } else {
            System.out.println(search + " does not exist in the list.");
        }

        sc.close();
    }
}
```

OUTPUT:-

Enter a number to search: 50

50 exists in the list.

Q12. Generic Container Create Container with add() and get() methods. Test it using Integer, String and Double.

```
class Container<T> {
    T value;

    void add(T value) {
        this.value = value;
    }

    T get() {
        return value;
    }
}
```

```
public class Q12 {
   public static void main(String[] args) {

      Container<Integer> c1 = new Container<>();
      Container<String> c2 = new Container<>();
      Container<Double> c3 = new Container<>();

      c1.add(100);
      c2.add("Hello");
      c3.add(45.5);

      System.out.println(c1.get());
      System.out.println(c2.get());
      System.out.println(c3.get());
   }
}
```

OUTPUT:-

```
100
Hello
45.5
```

Q13. Generic Class with Three Parameters Create Student where T = ID, U = Name and V = Marks. Create at least three student objects with suitable types.

```
class Student<T, U, V> {
   T id;
   U name;
   V marks;
```

```
    Student(T id, U name, V marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    void display() {
        System.out.println(id + " - " + name + " - " + marks);
    }
}

public class Q13 {
    public static void main(String[] args) {

        Student<Integer, String, Double> s1 =
                new Student<>(1, "Aarav", 85.5);

        Student<Integer, String, Double> s2 =
                new Student<>(2, "Riya", 91.0);

        Student<Integer, String, Double> s3 =
                new Student<>(3, "Kabir", 78.5);

        s1.display();
        s2.display();
        s3.display();
    }
```

}

OUTPUT:-

1 - Aarav - 85.5

2 - Riya - 91.0

3 - Kabir - 78.5

Q14. Generic Method and ArrayList Create an ArrayList and a generic method that displays all elements of the list. Test it with ArrayList and ArrayList.

```
import java.util.ArrayList;


public class Q14 {


  public static <T> void displayList(ArrayList<T> list) {


    for (T item : list) {

      System.out.println(item);

    }

  }


  public static void main(String[] args) {


    ArrayList<Integer> numbers = new ArrayList<>();


    numbers.add(10);

    numbers.add(20);

    numbers.add(30);


    ArrayList<String> names = new ArrayList<>();
```

```
        names.add("Aarav");
        names.add("Riya");
        names.add("Kabir");

        System.out.println("Integer list:");

        displayList(numbers);

        System.out.println("String list:");

        displayList(names);
    }
}
```

OUTPUT:-

```
Integer list:
10
20
30
String list:
Aarav
Riya
Kabir
```

Q15. Mini Generic Student Management Program Create a small student management program using Student, where T = Student ID, U = Student Name and V = Marks. Create at least five students, store them using an appropriate collection, display all students, search for a student, display the total number of students, and demonstrate type safety. Use Generic Class, Multiple Type Parameters, ArrayList, add(), get(), size(), contains(), loops, methods, constructors and type safety.

```
import java.util.ArrayList;
import java.util.Scanner;
```

```
class Student<T, U, V> {

    T id;
    U name;
    V marks;

    Student(T id, U name, V marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    void display() {
        System.out.println("ID: " + id +
                ", Name: " + name +
                ", Marks: " + marks);
    }
}

public class Q15 {

    public static void main(String[] args) {

        ArrayList<Student<Integer, String, Double>> students =
                new ArrayList<>();
```

```
ArrayList<Integer> studentIds = new ArrayList<>();

students.add(new Student<>(101, "Aarav", 85.5));
students.add(new Student<>(102, "Riya", 91.0));
students.add(new Student<>(103, "Kabir", 78.5));
students.add(new Student<>(104, "Meera", 88.0));
students.add(new Student<>(105, "Anaya", 95.0));

for (Student<Integer, String, Double> s : students) {
   studentIds.add(s.id);
}

System.out.println("All Students:");

for (int i = 0; i < students.size(); i++) {
   students.get(i).display();
}

Scanner sc = new Scanner(System.in);

System.out.print("Enter student ID to search: ");

int searchId = sc.nextInt();

if (studentIds.contains(searchId)) {

   System.out.println("Student ID " + searchId + " found.");
```

```
        for (Student<Integer, String, Double> s : students) {

            if (s.id.equals(searchId)) {
                s.display();
                break;
            }
        }

    } else {
        System.out.println("Student ID " + searchId + " not found.");
    }

    System.out.println("Total students: " + students.size());

    sc.close();
  }
}
```

OUTPUT:-

All Students:

ID: 101, Name: Aarav, Marks: 85.5

ID: 102, Name: Riya, Marks: 91.0

ID: 103, Name: Kabir, Marks: 78.5

ID: 104, Name: Meera, Marks: 88.0

ID: 105, Name: Anaya, Marks: 95.0

Enter student ID to search: 103

Student ID 103 found.

ID: 103, Name: Kabir, Marks: 78.5

Total students: 5